PUCPR · 2026.2

Projeto Final CSCF

Estufa Inteligente: três sistemas ciberfísicos integrados por MQTT

Disciplina: Conectividade de Sistemas Ciberfísicos
Ambiente: Wokwi · ESP32 · MicroPython · Broker MQTT público (HiveMQ)

Equipe

João Francisco Zupeli
Gustavo Vilela
João Dib

Entregas (links)

Link Wokwi 1 (CPS 1): _____

Link Wokwi 2 (CPS 2): _____

Link Wokwi 3 (CPS 3): _____

Link do dashboard: _____

Identificador da equipe nos tópicos MQTT: `cscf/zupeli_vilela_dib/`

1. A ideia do projeto

A gente montou uma **estufa inteligente**. Em vez de fazer um sistema só gigante, dividimos a estufa em três sistemas ciberfísicos pequenos, cada um com seu ESP32, seu sensor e seus atuadores, rodando em um projeto Wokwi separado. Cada um funciona sozinho, mas eles conversam entre si pela rede usando MQTT, e um dashboard no navegador mostra tudo e deixa a gente mandar comandos.

A graça é que o que um sistema percebe muda o que o outro faz. Se esquenta demais, o CPS 1 avisa; o CPS 2 abre a janela; o CPS 3 percebe que com calor e janela aberta o solo seca mais rápido e passa a regar mais cedo. Ninguém fala direto com ninguém, tudo passa pelo broker.

Resumo dos três sistemas

Sistema	Sensor	Atuadores	Papel na estufa
CPS 1 · Estação climática	DHT22 (temperatura e umidade do ar), GPIO 15	LED vermelho (alerta de calor), GPIO 2 LED azul (irrigando), GPIO 4	Mede o clima e avisa quando está quente demais.
CPS 2 · Ventilação e iluminação	LDR / fotoresistor (luminosidade), GPIO 34	Servo motor = janela, GPIO 18 LED branco = iluminação, GPIO 5	Abre a janela quando tem calor e acende a luz à noite.
CPS 3 · Irrigação	Potenciômetro = umidade do solo, GPIO 35	Módulo relé = bomba d'água, GPIO 26 LED verde, GPIO 27	Decide quando regar, levando em conta o clima.

2. Como cada sistema funciona

Os três programas têm a mesma estrutura (é mais fácil de explicar e de depurar): conectam no Wi-Fi, conectam no broker, assinam seus tópicos e ficam num laço que recebe mensagens (`check_msg`), lê o sensor a cada 3 segundos, decide, aciona o atuador e publica.

CPS 1 · Estação climática

O DHT22 lê temperatura e umidade. Se a temperatura passa do limite (30 °C por padrão, dá para mudar pelo dashboard; tem histerese de 1 °C para não ficar oscilando), o sistema entende que há **CALOR**, acende o LED vermelho e publica o alerta em `cps1/alerta`. Ele também assina `cps3/estado`: quando a bomba do CPS 3 está ligada, acende o LED azul. Ou seja, o atuador do CPS 1 também depende de um dado que veio de outro sistema.

CPS 2 · Ventilação e iluminação

O LDR mede a luz e o programa converte para porcentagem. Abaixo de 30% é "noite" e o LED branco acende (essa decisão é local; histerese até 35%). Além disso o CPS 2 assina `cps1/alerta`: se chegar **CALOR**, o servo vai para 90° (janela aberta); quando volta ao **NORMAL**, fecha (0°). O estado da janela é publicado em `cps2/estado`, que o CPS 3 usa.

CPS 3 · Irrigação

O potenciômetro simula a umidade do solo (0 a 100%). A bomba (relé + LED verde) liga quando o solo está mais seco que o **limite efetivo** e só desliga 10 pontos acima dele (histerese). O limite efetivo é calculado com dados da rede:

- limite base = 30% (ajustável pelo dashboard);
- + 20 pontos se o CPS 1 avisou CALOR (`cps1/alerta`);
- + 10 pontos se a janela do CPS 2 está aberta (`cps2/estado`), porque o ar fica mais seco.

Com calor e janela aberta o limite vai a 60%: a bomba liga mesmo com o solo "razoável". Depois o CPS 3 publica `cps3/estado` e o CPS 1 acende o LED azul.

Modo automático e manual

Todo comando vindo do dashboard coloca aquele sistema em modo `manual` (ele obedece o botão). O botão **Automático** do dashboard devolve a decisão para a lógica local.

3. O broker MQTT

Usamos o broker público **HiveMQ** (`broker.hivemq.com`), o mesmo dos exemplos do Wokwi usados nas aulas.

Quem conecta	Endereço	Observação
ESP32 (MicroPython, `umqtt.simple`)	`broker.hivemq.com`, porta **1883** (TCP, sem TLS)	O Wokwi dá internet pelo Wi-Fi simulado `Wokwi-GUEST`.
Dashboard (mqtt.js no navegador)	`wss://broker.hivemq.com:8884/mqtt`	Navegador não abre TCP puro, então usa MQTT sobre WebSocket seguro.

Recursos do MQTT que usamos:

- **Publish/Subscribe** com tópicos hierárquicos e o curinga `#` no dashboard.
- **Mensagens retidas** (`retain`) em alerta, estado e status: quem conecta depois (ou reconecta) já recebe o último valor.
- **Last Will**: cada ESP32 deixa registrado `offline` em `.../status`; se ele cair sem avisar, o próprio broker publica isso.
- **Keep-alive** de 30 s e `ping` periódico para detectar conexão morta.
- **QoS 0**: tá bom para o caso, porque o sensor é republicado a cada 3 s e os estados são retidos.

Limitação honesta: é um broker público e sem senha, então qualquer pessoa que souber o prefixo dos tópicos lê e envia mensagens. Por isso o prefixo é único (`zupeli_vilela_dib`). Num sistema real usaríamos usuário/senha e TLS.

4. Tabela de tópicos MQTT

Todos os tópicos começam com o prefixo `cscf/zupeli_vilela_dib/` (escrito só uma vez abaixo para a tabela caber). Todas as mensagens são JSON em UTF-8, exceto `status`, que é texto simples.

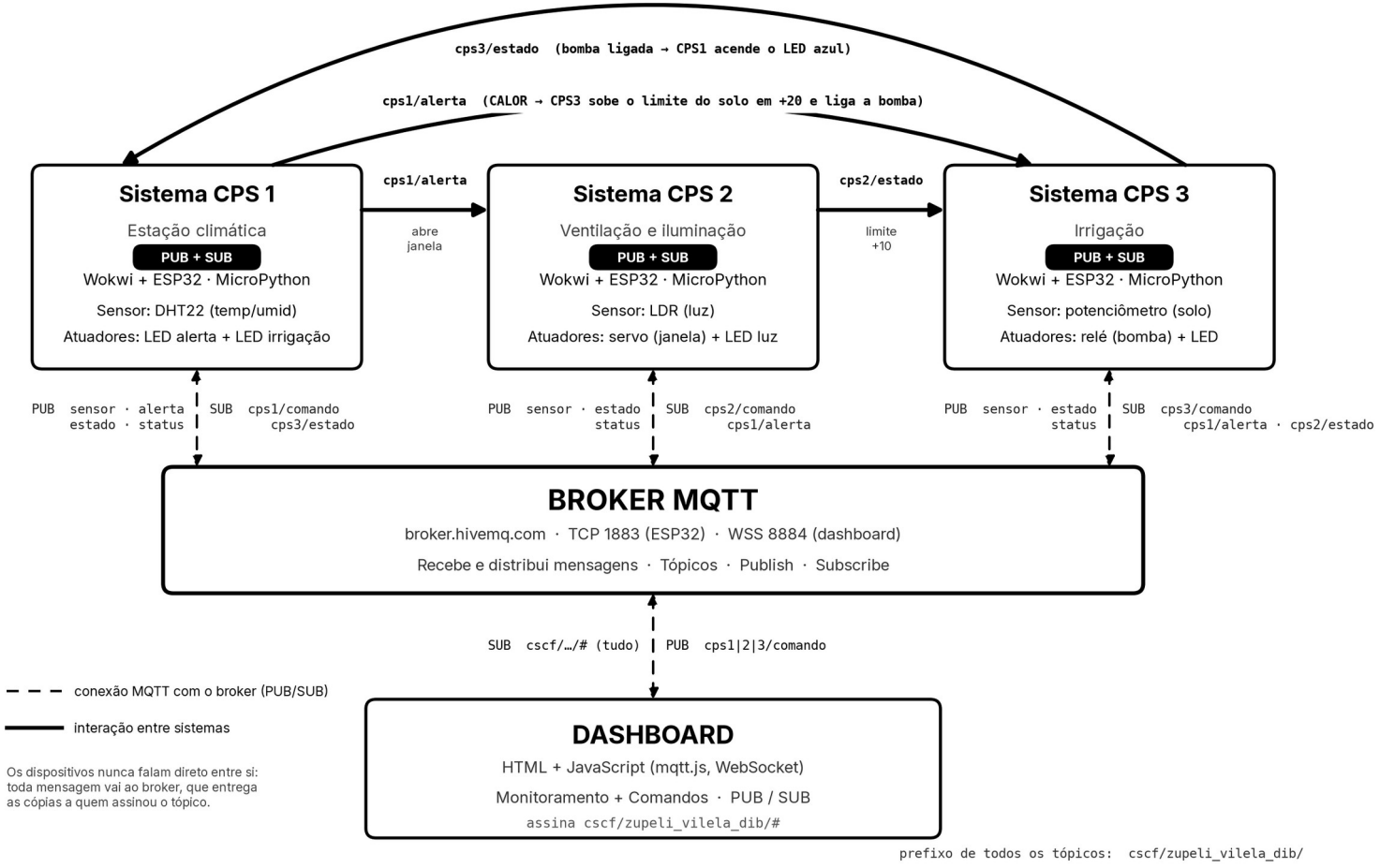
Tópico	Publicador	Assinantes	Tipo de mensagem / exemplo
`cps1/sensor`	CPS 1	Dashboard	JSON a cada 3 s `{"sistema":"CPS1","temp":24.0,"umid":60.0,"limite":30.0,"alerta":false}`
`cps1/alerta` (retida)	CPS 1	**CPS 2**, **CPS 3**, Dashboard	JSON só quando muda `{"sistema":"CPS1","condicao":"CALOR","temp":35.0,"limite":30.0}`
`cps1/estado` (retida)	CPS 1	Dashboard	JSON só quando muda `{"sistema":"CPS1","modo":"auto","led_alerta":"on","led_irrig":"off"}`
`cps1/status` (retida)	CPS 1 (Last Will)	Dashboard	texto: `online` / `offline`
`cps1/comando`	Dashboard	CPS 1	JSON `{"modo":"manual","led_alerta":"on","limite":28}`
`cps2/sensor`	CPS 2	Dashboard	JSON a cada 3 s `{"sistema":"CPS2","luz":72,"periodo":"dia"}`
`cps2/estado` (retida)	CPS 2	**CPS 3**, Dashboard	JSON só quando muda `{"sistema":"CPS2","modo":"auto","janela":"aberta","angulo":90,"luz":"off","motivo":"alerta de calor do CPS1"}`
`cps2/status` (retida)	CPS 2 (Last Will)	Dashboard	texto: `online` / `offline`
`cps2/comando`	Dashboard	CPS 2	JSON `{"modo":"manual","janela":"abrir","luz":"on"}`
`cps3/sensor`	CPS 3	Dashboard	JSON a cada 3 s `{"sistema":"CPS3","solo":53,"limite_base":30,"limite_efetivo":60}`
`cps3/estado` (retida)	CPS 3	**CPS 1**, Dashboard	JSON só quando muda `{"sistema":"CPS3","modo":"auto","bomba":"ligada","limite_efetivo":60,"motivo":"limite 60%: calor no CPS1 (+20) e janela aberta no CPS2 (+10)"}`
`cps3/status` (retida)	CPS 3 (Last Will)	Dashboard	texto: `online` / `offline`
`cps3/comando`	Dashboard	CPS 3	JSON `{"modo":"manual","bomba":"on","limite_solo":40}`

Visão por dispositivo

Dispositivo	Publica	Assina
CPS 1 (DHT22 + LEDs)	`cps1/sensor`, `cps1/alerta`, `cps1/estado`, `cps1/status`	`cps1/comando`, `cps3/estado`
CPS 2 (LDR + servo + LED)	`cps2/sensor`, `cps2/estado`, `cps2/status`	`cps2/comando`, `cps1/alerta`
CPS 3 (solo + relé + LED)	`cps3/sensor`, `cps3/estado`, `cps3/status`	`cps3/comando`, `cps1/alerta`, `cps2/estado`
Dashboard	`cps1/comando`, `cps2/comando`, `cps3/comando`	`cscf/zupeli_vilela_dib/#` (tudo)

5. Arquitetura de conectividade

O diagrama abaixo mostra os três ESP32 (cada um com sensor e atuador, publicando e assinando), o broker no meio e o dashboard. As setas tracejadas são as conexões MQTT de cada dispositivo com o broker; as setas pretas grossas são as interações entre os sistemas (que também passam pelo broker, o desenho só mostra a relação lógica).



6. A integração entre os três sistemas

O ponto principal do trabalho: não são três projetos mandando dado para o dashboard. Os sistemas usam as mensagens uns dos outros para decidir. A cadeia é essa:

1. O CPS 1 mede temperatura ≥ 30 °C, decide que é **CALOR**, acende o LED vermelho e publica `cps1/alerta = CALOR` (retida).
2. O CPS 2 recebe o alerta pelo broker e **abre a janela** (servo vai a 90°). Em seguida publica `cps2/estado = aberta`.
3. O CPS 3 recebe os dois (alerta do CPS 1 e estado da janela do CPS 2), calcula o limite efetivo (30 + 20 + 10 = 60%). Como o solo está abaixo disso, **liga a bomba** (relé + LED verde).
4. O CPS 3 publica `cps3/estado = ligada` e o CPS 1 recebe e **acende o LED azul**. É a realimentação: o primeiro sistema "fica sabendo" do efeito da cadeia.
5. Quando a temperatura cai, o CPS 1 publica `NORMAL`: a janela fecha, o limite volta a 30%, a bomba desliga e os LEDs apagam.

Fluxo	Tópico	O que muda no assinante
CPS 1 → CPS 2	`cps1/alerta`	CALOR abre a janela (servo 90°); NORMAL fecha.
CPS 1 → CPS 3	`cps1/alerta`	CALOR aumenta em 20 pontos o limite de umidade do solo.
CPS 2 → CPS 3	`cps2/estado`	Janela aberta aumenta em mais 10 pontos o limite.
CPS 3 → CPS 1	`cps3/estado`	Bomba ligada acende o LED azul do CPS 1.

Detalhe de implementação: alerta e estado só são publicados quando **mudam** (o sensor sim, a cada 3 s), e cada sistema reage na hora dentro do callback da mensagem. Isso evita um laço infinito de mensagens entre os sistemas.

Tratamento de falhas e reconexão

- **Wi-Fi:** o laço principal checa `wlan.isconnected()`; se caiu, reconecta antes de qualquer coisa.
- **MQTT:** qualquer erro em `check_msg`/`publish` cai num `try/except`, o cliente é fechado e o programa tenta reconectar com espera crescente (2 s, 4 s... até 10 s). Ao reconectar, assina os tópicos de novo e republica o estado.
- **Mensagens inválidas:** JSON quebrado ou campo desconhecido é ignorado, o programa não cai.
- **Last Will:** se um ESP32 cair, o dashboard mostra `offline`.

7. O ciclo ciberfísico

Etapa	Como aparece no projeto
Aquisição	DHT22 (temperatura e umidade), LDR (luz) e potenciômetro no ADC (umidade do solo) leem o "ambiente" simulado no Wokwi.
Processamento	Conversão para unidades, limites com histerese, classificação dia/noite e cálculo do limite efetivo de irrigação com dados de outros sistemas.
Conectividade	JSON sobre MQTT via broker HiveMQ: sensores, alertas, estados e comandos.
Atuação	LEDs, servo (janela) e relé (bomba), influenciados por dado local, por mensagens de

Etapa	Como aparece no projeto
	outros sistemas e por comandos do dashboard.
Realimentação	O estado de cada atuador volta para o broker (retido) e é usado pelos outros CPS e pelo dashboard, fechando o ciclo.

8. Dashboard

É um arquivo só (`dashboard/index.html`), HTML + CSS + JavaScript, que usa a biblioteca mqtt.js (carregada por CDN) e conecta no broker por WebSocket seguro. Para usar, é só abrir no navegador (ou publicar em qualquer hospedagem estática). Ele:

- assina `cscf/zupeli_vilela_dib/#` e atualiza em tempo real os sensores dos três sistemas, o estado dos atuadores, o modo (auto/manual) e se cada um está online/offline;
- tem botões que publicam comandos JSON em `cps1/comando`, `cps2/comando` e `cps3/comando` (LED de alerta, limite de calor; janela e luz; bomba e limite do solo; botão Automático);
- mostra a **cadeia de integração** CPS 1 → CPS 2 → CPS 3 → CPS 1 acendendo conforme as mensagens chegam;
- tem um painel de **alertas e interações entre sistemas** (log com hora de cada mudança: calor detectado, janela aberta, bomba ligada) e um log de todas as mensagens MQTT recebidas e enviadas.

Se outra equipe usar o mesmo prefixo as mensagens se misturam. Dá para trocar o identificador com `index.html?equipe=outroid` (e nos três `main.py`, na constante `EQUIPE`).

9. Perguntas da demonstração

Qual é a função do broker MQTT e como ele encaminha as mensagens?

O broker é o intermediário do modelo publish/subscribe. Cada cliente (os três ESP32 e o dashboard) abre uma conexão só com ele. Quando alguém publica uma mensagem em um tópico, o broker olha quem assinou aquele tópico e entrega uma cópia para cada assinante. Também guarda a última mensagem retida e dispara o Last Will quando um cliente some.

Qual é a diferença entre publicar e assinar um tópico?

Publicar é **enviar** uma mensagem para um tópico (ex.: o CPS 1 publica o alerta em `cps1/alerta`). Assinar é **pedir ao broker** para receber tudo o que for publicado naquele tópico (ex.: o CPS 2 assina `cps1/alerta`). Quem publica não sabe quem está ouvindo, e quem assina não sabe quem publicou.

Quais tópicos são usados e quem publica e assina cada um?

Estão na tabela da seção 4. Resumindo: cada CPS publica seus `sensor`, `estado` e `status` (o CPS 1 também publica `alerta`) e assina o próprio `comando` mais os tópicos de outros sistemas que influenciam o seu atuador (CPS 1 assina `cps3/estado`; CPS 2 assina `cps1/alerta`; CPS 3 assina `cps1/alerta` e `cps2/estado`). O dashboard assina tudo (`#`) e publica nos três `comando`.

Como uma mensagem enviada por um sistema chega aos demais?

Exemplo: o CPS 1 publica `CALOR` em `cps1/alerta` pelo `MQTTClient.publish`. A mensagem vai ao broker HiveMQ, que vê que CPS 2, CPS 3 e dashboard assinaram esse tópico e manda uma cópia para cada um. No CPS 2 e no CPS 3 o `check_msg()` do laço principal chama a função `ao_receber`, que lê o JSON e reage.

Como a informação recebida pela rede é processada e usada para acionar um atuador?

A função `ao_receber` converte o JSON (`ujson.loads`), confere o tópico e atualiza uma variável de estado (por exemplo `calor_cps1 = True` no CPS 2). Logo depois chama a função de decisão: no CPS 2, `calor_cps1` verdadeiro faz o servo ir para 90° via PWM; no CPS 3, a informação entra no cálculo do limite efetivo e o relé liga se o solo estiver abaixo; no CPS 1, `bomba: ligada` acende o LED azul.

Como o dashboard participa da comunicação MQTT?

Ele é mais um cliente MQTT, só que no navegador (mqtt.js por WebSocket seguro). Assina `cscf/zupeli_vilela_dib/#` para mostrar sensores e estados e publica comandos nos tópicos `.../comando`, que os ESP32 recebem como qualquer outra mensagem. Não existe nenhum canal fora do MQTT: o que o dashboard mostra é o que chegou pelo broker.

Quais as vantagens do MQTT em sistemas ciberfísicos distribuídos?

- Protocolo leve, bom para microcontroladores como o ESP32 e para redes ruins.
- Desacoplamento: dá para adicionar um quarto sistema só assinando tópicos, sem mexer nos outros.
- Um-para-muitos nativo: uma publicação chega a vários assinantes.
- Mensagens retidas e Last Will ajudam a manter o estado e detectar falhas.
- Tópicos hierárquicos organizam bem sensores, estados e comandos.
- Funciona sobre a internet comum (e WebSocket para navegadores).

10. Roteiro do vídeo de demonstração (6 a 8 minutos)

Antes de gravar: abrir em abas separadas os três projetos Wokwi e o dashboard, rodar os três (▶) e conferir os três "online" no dashboard. Gravar a tela inteira, falar devagar e mostrar sempre a mensagem MQTT correspondente no log do dashboard.

Tempo	O que mostrar	Quem fala / o que dizer
0:00	Capa e diagrama de arquitetura	Apresentar o grupo e a ideia: estufa com 3 CPS ligados por MQTT.
0:40	Broker HiveMQ e tabela de tópicos	Broker recebe e distribui; os dispositivos só conhecem o broker; prefixo único da equipe.
1:20	Wokwi 1: diagrama + `main.py`	DHT22, LEDs; mostrar `conectar_wifi`, `conectar_mqtt` (Last Will), `ao_receber` e o laço com reconexão.
2:10	Wokwi 2	LDR, servo e LED; assina `cps1/alerta`.
2:50	Wokwi 3	Potenciômetro, relé e LED; assina `cps1/alerta` e `cps2/estado`; limite efetivo.

Tempo	O que mostrar	Quem fala / o que dizer
3:30	Dashboard com os três online	Valores chegando por MQTT; mostrar o log de mensagens.
4:00	**Integração:** no Wokwi 1 subir a temperatura do DHT22 para 35 °C	CPS 1 detecta CALOR e publica `cps1/alerta`.
4:30	Servo do Wokwi 2 gira; relé/LED do Wokwi 3 liga; LED azul do Wokwi 1 acende; dashboard mostra a cadeia e o painel de interações	Explicar a cadeia CPS 1 → CPS 2 → CPS 3 → CPS 1.
5:20	Baixar a temperatura para 24 °C	Volta a NORMAL: janela fecha, bomba desliga, LEDs apagam.
5:40	**Comandos do dashboard:** Abrir janela (CPS 2), Bomba ON (CPS 3), LED de alerta (CPS 1), depois "Automático"	Dashboard publica em `.../comando`; mostrar cada atuador reagindo no Wokwi.
6:30	Girar o LDR para pouca luz	CPS 2 vira "noite" e acende a iluminação (decisão local).
6:50	Parar o Wokwi 1 e depois iniciar de novo	Aparece `offline` (Last Will) e depois `online`: reconexão.
7:20	Encerramento	Resumir: aquisição, processamento, conectividade, atuação e realimentação; vantagens do MQTT.

11. Como testamos

- Os três `main.py` foram executados em MicroPython (versão unix) com módulos simulados de hardware, contra um broker Mosquitto local: publicação, assinatura, a cadeia CPS 1 → CPS 2 → CPS 3 → CPS 1, comandos do dashboard, mensagens inválidas, queda do Wi-Fi e queda do broker (teste em `testes/teste_integracao.py`, tudo passou).
- O dashboard foi aberto num Chrome real, recebeu mensagens MQTT publicadas por um cliente paho-mqtt, atualizou os valores e o painel de interações, e os botões publicaram os comandos esperados nos tópicos corretos.
- Os `diagram.json` foram validados como JSON e as conexões conferidas pino a pino.

O que **não** foi possível aqui: rodar dentro do Wokwi online e contra o HiveMQ de verdade. Por isso, ao montar os projetos, conferir se o terminal mostra "Wi-Fi conectado" e "MQTT conectado" (se faltar o módulo, adicionar `umqtt.simple` no Library Manager).